

1113: Modern Java Development Workflow: Quarkus, Containers, and IBM Bob

Hands-on lab guide

Author name

Markus Eisele (markus.eisele@ibm.com)

Alex Soto (asotobue@ibm.com)

IBM Developer Advocates

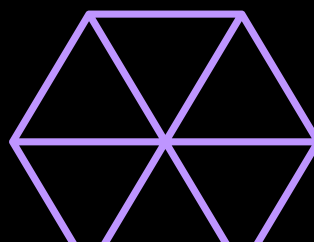


Table of Contents

1	LAB-1113: Build, persist, and containerize a Quarkus app with IBM Bob.....	4
2	What you will have at the end	5
3	How Bob, MCP, and you divide the work	5
4	Time budget.....	5
5	Terms you will use.....	6
6	Check the lab environment	6
6.1	Install or verify IBM Bob	6
6.2	Sign in with IBMid	7
6.3	Confirm the toolchain versions.....	7
6.4	Open an empty workspace	8
7	Configure the Quarkus Agent MCP	9
7.1	Enable MCP in Bob	9
7.2	Add the Quarkus Agent server.....	9
7.3	Review tool approvals.....	11
7.4	Add AGENTS.md for Quarkus MCP.....	11
7.5	Checkpoint: list the tools	13
8	Create the REST service	13
8.1	Create greeting-service	14
8.2	Hit the hello endpoint	15
8.3	Add a JSON greeting endpoint	15
9	Add persistence.....	16
9.1	Persist greetings through Panache	17
9.2	Review the shape of the code.....	18
9.3	Verify POST then GET	19

IBM TechXchange

10	Containerize with Podman	20
10.1	Build the image and prepare prod config	21
10.2	Review prod configuration	22
10.3	Run Postgres and the app	23
10.4	Prove the containerized API	23
10.5	Optional: Plan a Qute UI	24
11	Close the loop	26
11.1	Recall	27
11.2	Artifacts	27
11.3	What changed	27
11.4	Further reading	28
11.5	Need help?	28

Notices and disclaimers

© 2026 International Business Machines Corporation. No part of this document may be reproduced or transmitted in any form without written permission from IBM.

U.S. Government Users Restricted Rights — use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM.

This document is current as of the initial date of publication and may be changed by IBM at any time. Not all offerings are available in every country.

Information in these presentations (including information relating to products that have not yet been announced by IBM) has been obtained from sources believed to be reliable, but IBM does not warrant or guarantee the accuracy or completeness of such information.

This document is distributed “as is” without any warranty, either express or implied. In no event, shall IBM be liable for any damages arising out of the use of this information.

IBM products are manufactured from new parts or new and used parts. In some cases, a product may not be new and may have been previously installed.

Any statements regarding IBM's future direction, intent or product plans are subject to change or withdrawal without notice.

IBM TechXchange

Performance data contained herein was generally obtained in a controlled, isolated environments. Customer examples are presented as illustrations of how those customers have used IBM products and the results they may have achieved.

References in this document to IBM products, programs, or services does not imply that IBM intends to make such products, programs, or services available in all countries in which IBM operates.

Workshops, sessions and associated materials may have been prepared by independent session speakers, and do not necessarily reflect the views or positions of IBM.

It is the customer's responsibility to ensure its own compliance with legal requirements and to obtain advice of competent legal counsel.

Notices and disclaimers (Continued)

Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products. IBM does not warrant the performance or compatibility of non-IBM products.

The provision of the information contained herein is not intended to, and does not, grant any right or license under any IBM patents or intellectual property.

IBM, the IBM logo, and ibm.com are trademarks of International Business Machines Corporation, registered in many jurisdictions worldwide.

1 LAB-1113: Build, persist, and containerize a Quarkus app with IBM Bob

Modern Java development is no longer just about writing code. It is about building, validating, and running applications in environments that match production from the start.

This lab brings those concerns together with **Quarkus** and **IBM Bob**. You use Bob to guide decisions, generate structure, and improve the codebase as it evolves. The Quarkus Agent MCP gives Bob the project lifecycle, extension patterns, and documentation lookup that a generic chat session does not have.

By the end, you will have a working Quarkus application running in a container, and a practical sense of how an AI-assisted workflow reduces friction between development and operations.

NOTE: Bob output varies

Class names, file paths, and port numbers can differ between runs. Verify **behavior**: endpoints respond, tests pass, data survives a second request, and the container serves the API. Do not require an exact diff match to the snippets in this tutorial.

2 What you will have at the end

- .bob/mcp.json with the Quarkus Agent MCP configured for this workspace
- AGENTS.md at the workspace root so Bob uses Quarkus MCP on every conversation
- A greeting-service Quarkus app created and operated through MCP tools
- REST endpoints that create and list greetings
- PostgreSQL persistence: Dev Services in development, a Postgres container next to the app image
- A Podman image localhost/greeting-service:lab serving the API

3 How Bob, MCP, and you divide the work

The naive approach is to paste Maven commands into chat and hope Bob guesses Quarkus patterns.

The working approach for this lab:

- **Quarkus Agent MCP** owns project create, dev mode, tests, exception lookup, and documentation search.
- ``quarkus_skills`` supplies extension-specific patterns. Bob must read skills **before** writing Quarkus code.
- **Bob** orchestrates bounded edits and tool calls.
- **You** approve tool use, review the plan and the files, and prove behavior with curl and Podman.

If Bob skips MCP and types `mvn quarkus:dev` or `mvn test` by hand, stop it and point it back at the tools. Raw Maven is reserved for packaging the container image after you stop dev mode.

4 Time budget

Step	Section	Mode	Time
0	Check the lab environment	—	10 min
1	Configure Quarkus Agent MCP and AGENTS.md	Agent	15 min
2	Create the REST service	Agent	15 min
3	Add persistence	Agent	22 min
4	Containerize with Podman	Agent	22 min
5	Wrap-up	—	6 min

Total: about 90 minutes. If you run long, skip the optional Qute stretch at the end of section 4.

5 Terms you will use

MCP (Model Context Protocol) is how Bob connects to external tools. For more information, see [Understanding MCP](#).

Quarkus Agent MCP is a local JBang server that exposes `quarkus_create`, `quarkus_skills`, `quarkus_searchDocs`, `quarkus_callTool`, and related tools. It stays up even if the app crashes.

Skills are extension-specific patterns and testing guidance. Call `quarkus_skills` before writing REST, Panache, or container-image code.

Dev Services start a database container automatically in *dev* and *test* when you add a JDBC extension and do **not** set a datasource URL. Quarkus does not start Dev Services in *prod*. Your container run must supply a real database.

Agent mode lets Bob edit files, run commands, and call MCP tools. Some Bob builds label this **Advanced**. The capability is the same.

AGENTS.md is the file Bob reads at the start of every conversation. It gives Bob persistent project instructions. For more information, see [Start a project with /init and AGENTS.md](#).

New Task (+) (called **New chat** in some builds) starts a fresh conversation and resets the context window. Use it after you add AGENTS.md, and when you switch from planning to implementation. For more information, see [Create a new context window](#).

6 Check the lab environment

Time: 10 minutes

Lab machines are pre-provisioned with IBM Bob, Podman, the Quarkus CLI, JBang, and SDKMAN (Java and Maven). You still sign in and confirm the versions on *this* machine.

The screenshots were captured in **IBM Bob 2.2.1 on macOS** with Quarkus Agent MCP 1.0.10. Linux lab machines have different window controls and paths; follow the stated values for your machine.

Use **Quarkus 3.39.5** throughout this lab. The lab machines have Maven dependencies for this version pre-populated. The Quarkus CLI version can differ; the create prompt below pins the application's platform BOM and Maven plugin to 3.39.5. Do not upgrade the application during the lab.

6.1 Install or verify IBM Bob

If Bob is already on the applications menu, launch it and continue with sign-in.

If Bob is missing or will not start, follow [Installing IBM Bob](#):

1. Download the installer for your OS and CPU from bob.ibm.com/download.
2. On Linux lab VMs, install the .rpm or .deb package for your distribution.

IBM TechXchange

3. Launch Bob from the applications menu or desktop shortcut.

6.2 Sign in with IBMid

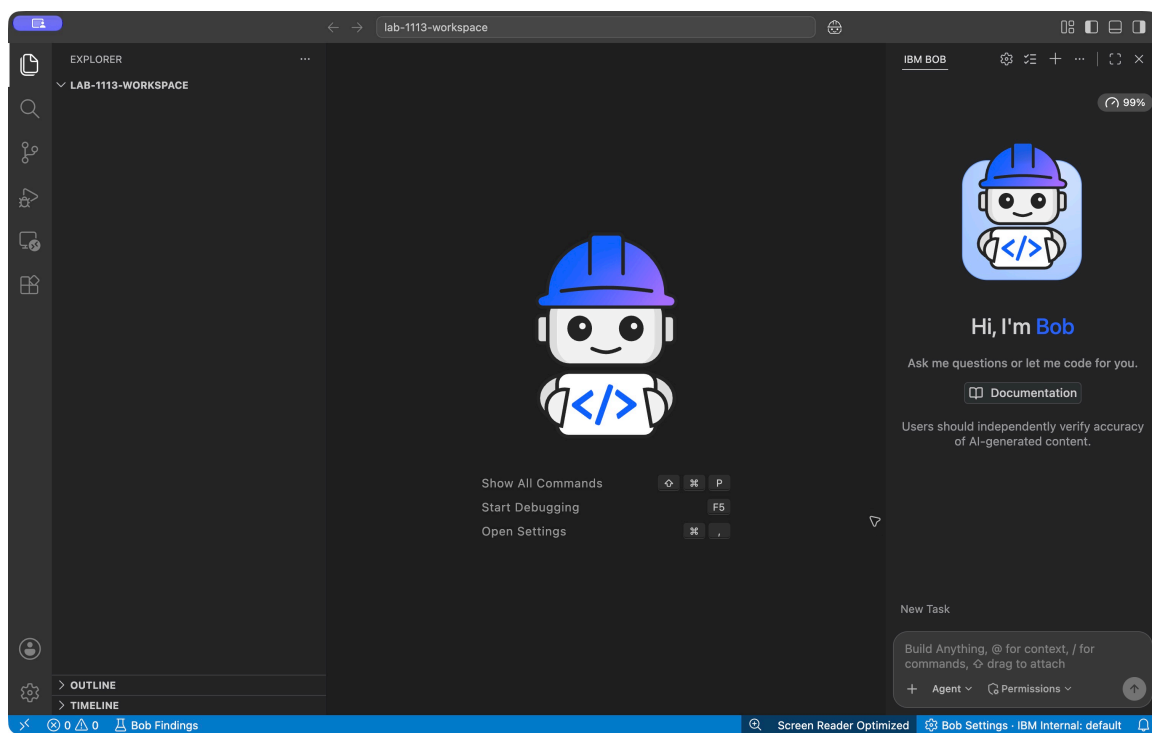
Sign-in is required before MCP configuration.

1. On first launch, complete the browser authentication flow with your **IBMid**.
2. Return to Bob after authentication succeeds.
3. If you do not have an IBMid, create one at [IBM account signup](#).

If sign-in hangs on the lab network, the environment might need to allow `login.ibm.com`. See [Configuring firewall rules](#) in the Bob docs.

SUCCESS: Stop and verify

Bob opens, the chat panel is available, and you are signed in (no repeated login prompt).



Bob IDE after sign-in

6.3 Confirm the toolchain versions

Open the **Bob integrated terminal**. Maven on lab machines is installed through **SDKMAN**. Source SDKMAN in this terminal before you check versions. On the Mac used for these screenshots, Maven is installed through Homebrew; `sdk current maven` can therefore report no selected version while `mvn -version` succeeds:

```
source "$HOME/.sdkman/bin/sdkman-init.sh"

java -version
mvn -version
sdk current java
sdk current maven
quarkus -v
jbang --version
podman --version
podman info
```

Lab images typically install Java through SDKMAN (for example IBM Semeru 25), Maven 3.9.x, a current Quarkus CLI, JBang, and Podman from the OS repositories. Your patch versions can differ.

Confirm:

- mvn -version reports Apache Maven 3.9.x (SDKMAN on the lab machines).
- sdk current maven shows the active Maven candidate.
- quarkus -v, jbang --version, and podman info all succeed.

Record two paths. You will paste them into MCP configuration in the next section:

```
which jbang
echo "$JAVA_HOME"
```

If JAVA_HOME is empty, print the SDKMAN Java home:

```
echo "$HOME/.sdkman/candidates/java/current"
```

SUCCESS: Stop and verify

Every command in the version block exits with code 0, and you have a full path to jbang plus a JAVA_HOME directory. If mvn is missing, see [Maven not found \(SDKMAN\)](#).

6.4 Open an empty workspace

Do the lab in a dedicated folder, not inside this documentation repository. The folder is empty on purpose. You have not created the app yet.

1. Create an empty folder, for example ~/lab-1113-workspace.
2. In Bob, choose **File → Open Folder** and select that folder.
3. Confirm the explorer shows an empty workspace. The next sections add MCP configuration, then AGENTS.md, then the Quarkus app.

If the chat panel is hidden, click the Bob icon in the navigation bar, or press **Option + Command + B** (macOS) / **Ctrl + Alt + B** (Windows/Linux). See the [Bob Quickstart](#).

7 Configure the Quarkus Agent MCP

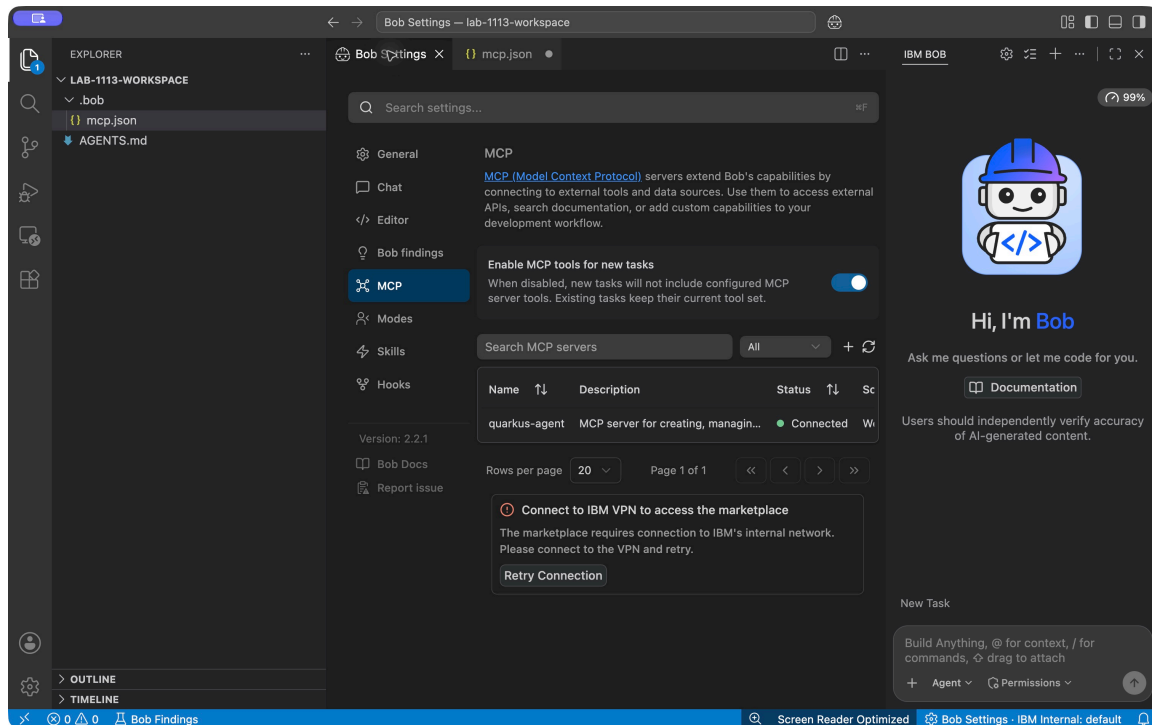
Time: 15 minutes **Mode:** Agent (or Advanced)

Bob does not ship Quarkus knowledge as a plugin. You connect a local MCP server that can create apps, start dev mode, read skills, search docs, and proxy Dev MCP tools such as the test runner.

7.1 Enable MCP in Bob

1. Click the **Settings** icon in the Bob panel.
2. Open the **MCP** tab.
3. Enable **Enable MCP tools for new tasks** (called **Use MCP Servers** in some earlier builds).

For more information, see [Using MCP in Bob](#).



MCP settings tab

7.2 Add the Quarkus Agent server

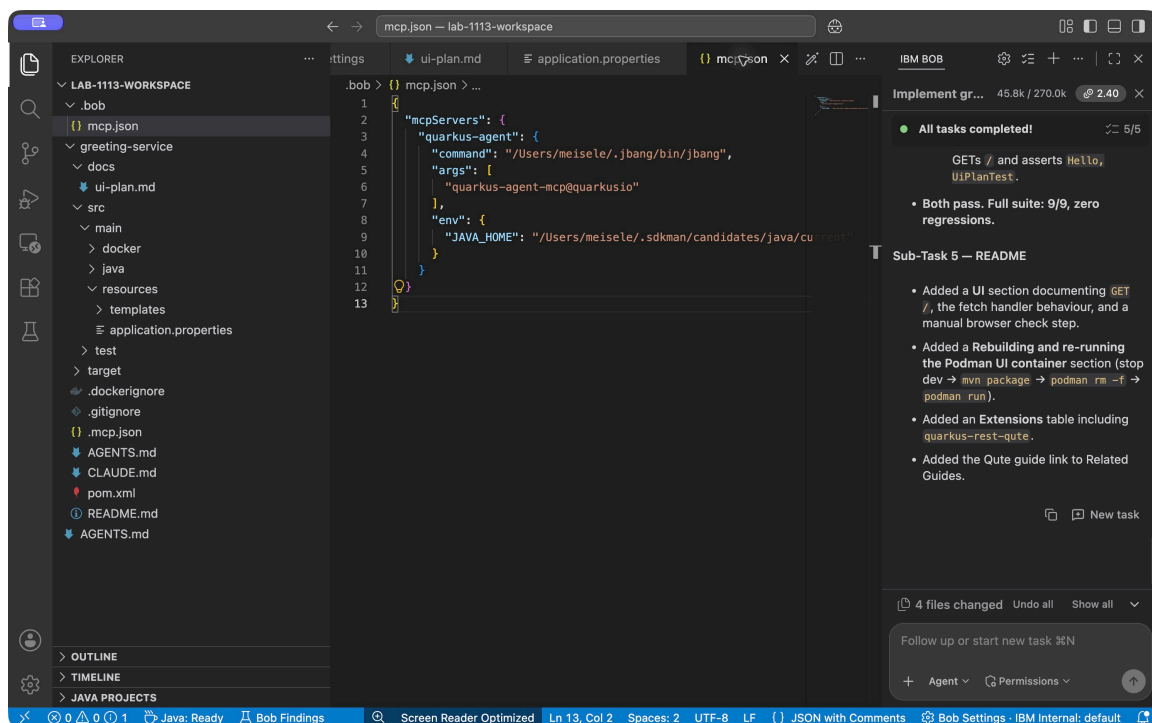
Bob launched from the desktop inherits the **system PATH**, not your shell PATH. SDKMAN installs JBang and Java on the shell PATH only, so command: "jbang" often fails with spawn jbang ENOENT. Use absolute paths.

1. In the MCP tab, click **Add MCP Server** and select the **lab-1113-workspace** configuration space. Click **Open Configuration File**. Bob creates .bob/mcp.json in the workspace if needed.

2. Replace the file contents with the following JSON. Substitute the paths you recorded from which jbang and echo "\$JAVA_HOME".

```
{
  "mcpServers": {
    "quarkus-agent": {
      "command": "/home/YOUR_USER/.sdkman/candidates/jbang/current/bin/jbang",
      "args": ["quarkus-agent-mcp@quarkusio"],
      "env": {
        "JAVA_HOME": "/home/YOUR_USER/.sdkman/candidates/java/current"
      }
    }
  }
}
```

Replace both /home/YOUR_USER/... example paths with the real absolute paths from your machine before Bob starts the server.



Project MCP JSON editor

3. Save the file.

4. In MCP settings you see the quarkus-agent server configured. Hit **refresh all servers** if it does not connect on its own.

5. Expand the server entry. Confirm tools such as quarkus_agent_log, quarkus_skills, quarkus_searchDocs, and quarkus_callTool appear.

IBM TechXchange

The first start can take a minute. JBang downloads quarkus-agent-mcp@quarkusio from Maven Central. Semantic doc search can also start a local documentation container through Podman. If search reports that version-specific documentation is unavailable, inspect the returned version and verify the result against the 3.39.5 build; do not upgrade the application to match the search index.

If the server stays red or tools never appear, see [MCP tools never appear](#) and [Quarkus Agent MCP server fails to start](#).

7.3 Review tool approvals

Bob 2.2.1 labels the control below the chat input **Permissions**. Earlier builds can label it **Auto-Approve**. Leave edit and command approvals enabled, review each proposed action, and choose **Approve once** when it matches the task. This walkthrough works with individual MCP approvals.

If you choose auto-approval, review the [Bob approval documentation](#) first. It skips the corresponding confirmation prompts. The lab does not require changing your existing permission policy.

7.4 Add AGENTS.md for Quarkus MCP

Bob is stateless between conversations. An AGENTS.md file in the **workspace root** is the persistent instruction file Bob loads at the start of each chat. Without it, later prompts have to repeat "use Quarkus MCP" and Bob can fall back to raw Maven.

Do **not** run /init in this empty folder. /init scans existing code; there is none yet, and it would not add the Quarkus MCP rules.

1. In the workspace root, create a file named AGENTS.md.
2. Paste the following contents into it.

```
# LAB-1113 – instructions for IBM Bob
```

This workspace is the lab root. Create the Quarkus application in a `greeting-service` subdirectory. Do all Quarkus work through the **quarkus-agent** MCP server.

```
## Tooling
```

- Use `quarkus\_create` to scaffold a new Maven Quarkus app. Set `quarkusVersion` to `3.39.5` and `noWrapper` to `true`; use the preinstalled system Maven. Do not run Maven archetypes or `quarkus create` in the terminal.
- After create or before any code change, call `quarkus\_skills` for the extensions you will touch.
- Use `quarkus\_searchDocs` for APIs, configuration, and current guide text. Do not invent Quarkus APIs from memory when a docs search exists.
- Use `quarkus\_start`, `quarkus\_stop`, `quarkus\_restart`, `quarkus\_status`, and `quarkus\_logs` for lifecycle.
- Discover Dev MCP tools with `quarkus\_searchTools`. Invoke them with `quarkus\_callTool`.
- Add extensions through Dev MCP extension tools. After dependency changes, stop and start through MCP, then discover tools again.

```
## Tests and failures
```

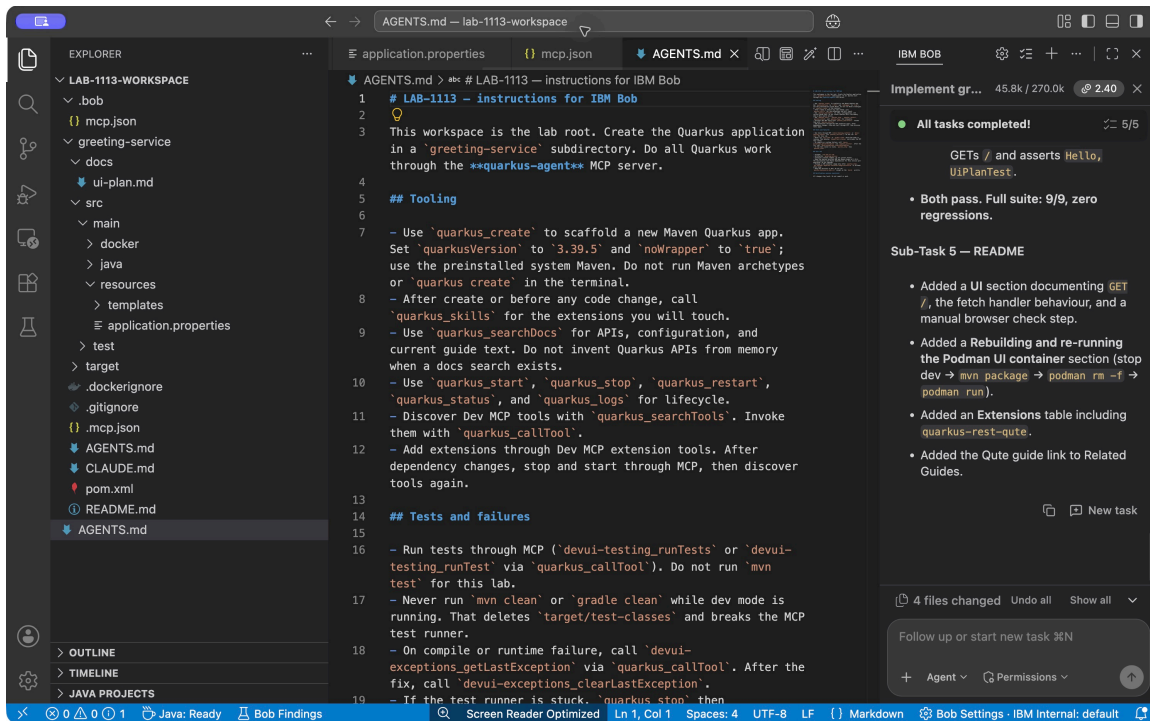
- Run tests through MCP (`devui-testing\_runTests` or `devui-testing\_runTest` via `quarkus\_callTool`). Do not run `mvn test` for this lab.
- Never run `mvn clean` or `gradle clean` while dev mode is running. That deletes `target/test-classes` and breaks the MCP test runner.
- On compile or runtime failure, call `devui-exceptions\_getLastException` via `quarkus\_callTool`. After the fix, call `devui-exceptions\_clearLastException`.
- If the test runner is stuck, `quarkus\_stop` then `quarkus\_start`.

```
## This lab
```

- groupId: `com.ibm.txc.lab`
- artifactId: `greeting-service`
- Container runtime: Podman. Do not install Docker.
- Keep the Quarkus platform BOM and Maven plugin at 3.39.5. The lab machines have Maven dependencies for this version pre-populated. Do not upgrade.
- Package a JVM container image only after `quarkus\_stop`. `mvn package -Dquarkus.container-image.build=true` is allowed after stop.
- Keep Dev Services in dev: do not set `quarkus.datasource.jdbc.url` except on the `%prod.` profile.

3. Save the file at `~/lab-1113-workspace/AGENTS.md` (or the empty folder you opened).

4. Click + (**New Task; New chat** in some builds) so the next conversation loads AGENTS.md. See [Start a project with /init and AGENTS.md](#).



Workspace-root AGENTS.md instructions

quarkus_create may later add a CLAUDE.md or AGENTS.md **inside** greeting-service/. Leave the workspace-root AGENTS.md in place. That is the file Bob reads for this lab.

7.5 Checkpoint: list the tools

Switch to **Agent** mode if you have not already.

Paste this prompt:

Read AGENTS.md in this workspace. List the Quarkus MCP tools you can use.
For each of quarkus_create, quarkus_skills, quarkus_searchDocs, and quarkus_callTool, explain in one sentence when you should call it in this lab.

You should see a tool list from the connected quarkus-agent server, not a generic guess. Bob should treat quarkus_skills as required **before** writing code, matching AGENTS.md.

SUCCESS: Stop and verify

quarkus-agent is connected, AGENTS.md is in the workspace root, and you can explain why skills come before code.

8 Create the REST service

Time: 15 minutes **Mode:** Agent (or Advanced)

IBM TechXchange

This is the first worked slice. Bob creates the app through MCP, the app starts in dev mode, and you prove the default endpoint with curl. After that, you ask for one JSON greeting endpoint and tests.

8.1 Create greeting-service

1. In the Bob integrated terminal, create an empty application directory under the workspace root:

```
mkdir -p greeting-service
```

2. Create a new task with + (**New Task**; called **New chat** in some earlier builds).
3. Paste this prompt:

```
Use the Quarkus MCP agent to create a new Maven Quarkus application in this workspace.

- artifactId: greeting-service
- groupId: com.ibm.txc.lab
- extensions: rest
- Quarkus version: 3.39.5 exactly (quarkusVersion: "3.39.5")
- Use the existing empty greeting-service subdirectory as outputDir
- Set createInCurrentDir: true
- Include starter code (not noCode)
- Set noWrapper: true; use the preinstalled system Maven
- After creation, call quarkus_skills for the REST extension before changing any code

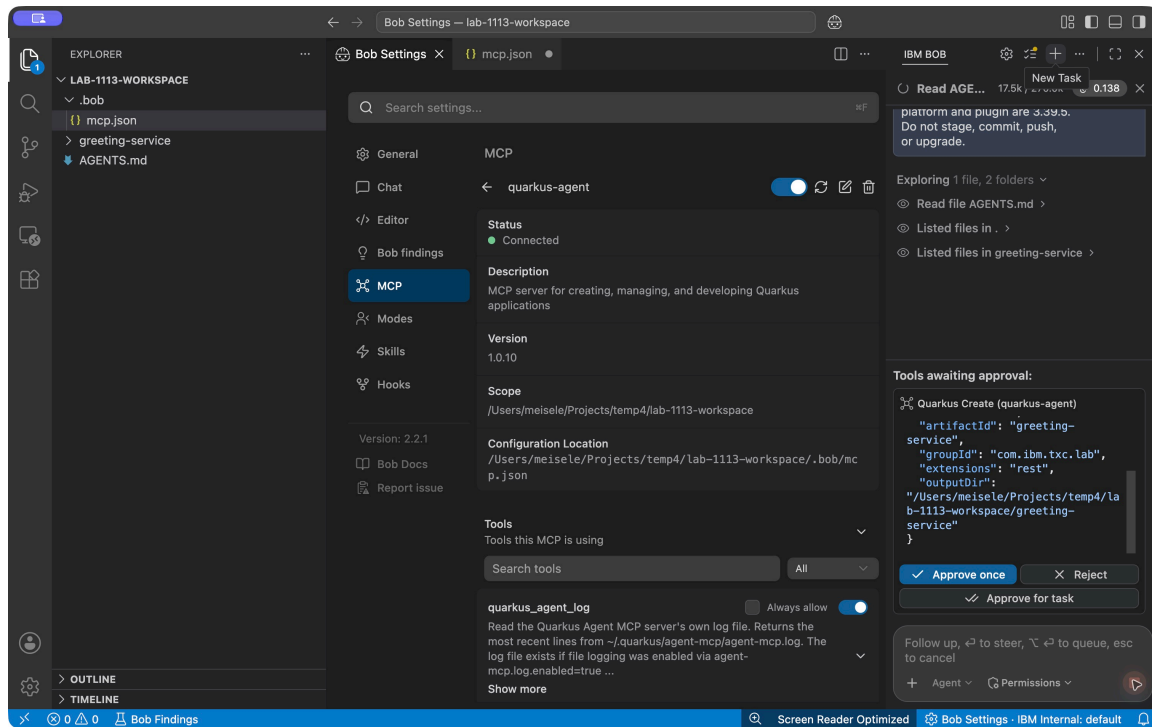
When the app is running, tell me the URL to hit for the default hello endpoint.
Do not run Maven create or quarkus:dev by hand. Use quarkus_create.
Do not upgrade Quarkus.
```

This lab uses system Maven because the MCP server can reject an untracked Maven wrapper in a new workspace. If creation reports that the output directory does not exist, check that greeting-service exists inside the workspace. See [Maven wrapper rejected by MCP](#).

Watch for:

1. Bob calls quarkus_create. Approve the MCP tool if prompted.
2. Bob presents a todo list. Approve if prompted.
3. Bob waits for the initial build and dev mode.
4. Bob calls quarkus_skills before suggesting further code changes.
5. The REST codestart exposes a default hello endpoint (often /hello).

Open greeting-service/pom.xml. Confirm quarkus.platform.version is 3.39.5 and that both the platform BOM and quarkus-maven-plugin use that property.



quarkus_create tool approval

If Bob reports a different port than 8080, use that port in every curl and browser URL for the rest of the lab. Dev mode can pick another port when 8080 is already bound.

8.2 Hit the hello endpoint

In the Bob terminal, call the URL Bob reported:

```
curl -i http://localhost:8080/hello
```

The output is similar to the following (path and body can differ):

```
HTTP/1.1 200 OK
content-length: 23
Content-Type: text/plain; charset=UTF-8

Hello from Quarkus REST
```

SUCCESS: Stop and verify

HTTP 200 and a response body from the REST codestart.

8.3 Add a JSON greeting endpoint

1. Start a new chat

2. This step fades the scaffolding: you specify the contract, not the class names. Paste:

```
Add a GET endpoint at /api/greeting?name=YourName that returns JSON:
{"message": "Hello, YourName"}.

Rules:
- Call quarkus_skills for REST before editing
- Add rest-jackson through MCP extension tools for JSON serialization
- After adding the extension, use quarkus_stop then quarkus_start
- Keep the Quarkus platform and Maven plugin at 3.39.5
- Add a test
- Run tests through Quarkus MCP testing tools (quarkus_searchTools + quarkus_callTool),
  not mvn test
- Do not add a database yet
```

3. Validate and approve tool calls as Bob asks for them.

Watch for: Bob uses quarkus_callTool with a testing tool such as devui-testing_runTests. If tests get stuck with "Tests already in progress", ask Bob to run quarkus_stop then quarkus_start through MCP. Do **not** run mvn clean while dev mode is running; it deletes target/test-classes and breaks the MCP test runner.

Verify the contract yourself:

```
curl "http://localhost:8080/api/greeting?name=Lab"
```

Expected shape (whitespace may differ):

```
{"message": "Hello, Lab"}
```

Open the resource class Bob created. You should see a Jakarta REST resource with a @QueryParam (or equivalent) and a JSON response. If the file looks unrelated to this contract, paste the curl output back into chat and ask Bob to match it.

SUCCESS: Stop and verify

The query parameter changes the JSON message, and MCP-run tests pass.

DANGER: Do not run mvn clean while dev mode is running

If tests fail, ask Bob to use devui-exceptions_getLastException via quarkus_callTool. See [Tests fail or runner stuck](#).

9 Add persistence

Time: 22 minutes **Mode:** Agent (or Advanced)

The JSON greeting so far is computed from the query parameter on every request; it is not stored. Next, store greetings in PostgreSQL and list them. Dev Services starts the development database so you do not configure a separate Postgres instance by hand.

Quarkus starts a PostgreSQL container through Podman when all of the following are true:

- A JDBC PostgreSQL extension is on the classpath.
- You have **not** set `quarkus.datasource.jdbc.url` in the default (dev/test) config.
- Podman (or Docker) is available.

That is Dev Services. It is the same idea described in the IBM Developer [Quarkus REST](#) material: you do not stand up Postgres by hand for quarkus dev.

Predict this before you paste the prompt: after a POST, a following GET should return the same greeting, including an id assigned by the database.

9.1 Persist greetings through Panache

1. Start a new chat
2. Paste this prompt:

```
Add PostgreSQL persistence to greeting-service.
```

```
Extensions:
```

- hibernate-orm-panache
- jdbc-postgresql

```
Behavior:
```

- POST `/api/greetings` with JSON `{"name": "Lab"}` creates a row and returns 201 with id, name, and message (message = "Hello, {name}")
- GET `/api/greetings` lists all stored greetings
- GET `/api/greetings/{id}` returns one greeting or 404

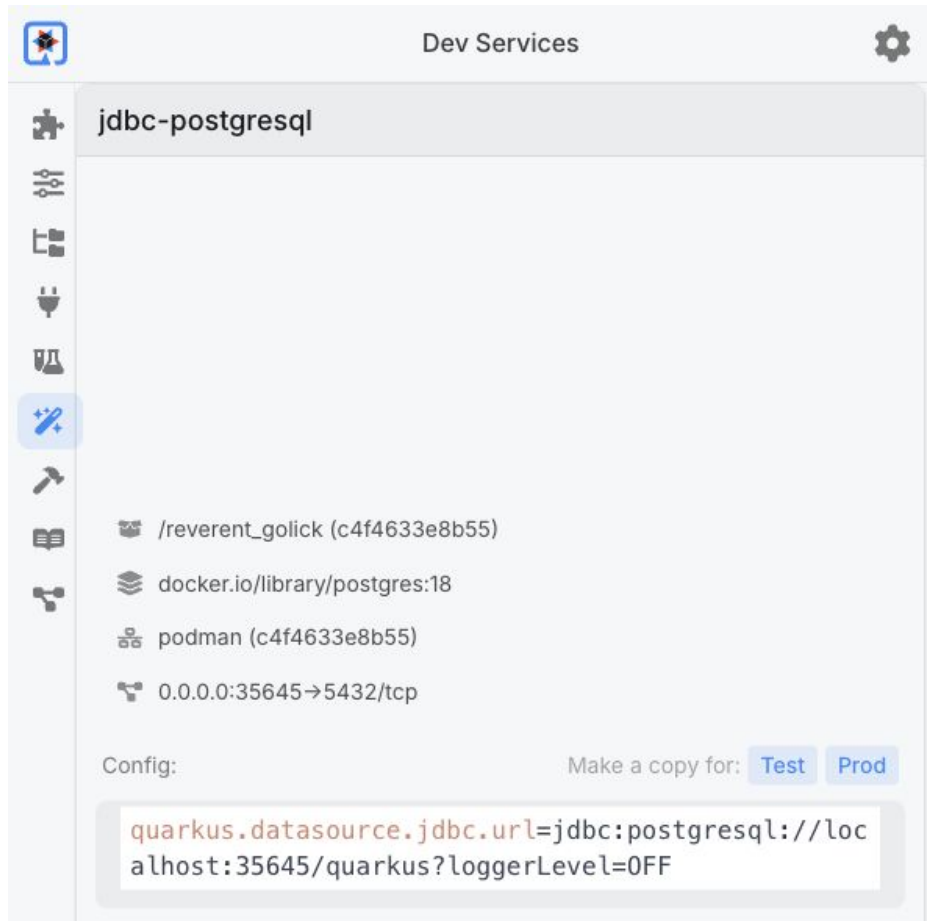
```
Rules:
```

- Call `quarkus_skills` separately for Panache, Hibernate ORM, and REST before editing
- Add extensions through MCP extension tools, then stop/start through MCP
- Keep the Quarkus platform and Maven plugin at 3.39.5
- Use a Panache entity (active record is fine)
- Annotate write operations with `@Transactional`
- Do NOT set `quarkus.datasource.jdbc.url` in the default config – use Dev Services
- Add or update tests and run them through Quarkus MCP tools
- After tests pass, tell me how to verify POST then GET with curl

Watch for:

- Bob adds extensions through MCP (`quarkus_callTool` / extension tools), not by editing `pom.xml` from memory.
- `quarkus_skills` runs **before** the entity and resource appear.
- Application logs mention a Dev Services PostgreSQL container.
- Writes use `@Transactional`. Without it, `persist()` fails or never commits.

3. Open the Quarkus Dev UI at <http://localhost:8080/q/dev-ui/dev-services>. Check that jdbc-postgresql has a running container and a generated JDBC URL. The Dev Services image version and port can differ from the explicit PostgreSQL 16 container used later.



Dev Services PostgreSQL started for greeting-service

9.2 Review the shape of the code

Bob's class names can differ. The structure should match this pattern from [Hibernate ORM with Panache](#):

```
@Entity
public class Greeting extends PanacheEntity {

    public String name;
    public String message;

    public static Greeting of(String name) {
        Greeting g = new Greeting();
        g.name = name;
        g.message = "Hello, " + name;
        return g;
    }
}
```

PanacheEntity provides the id field. Public fields are the Panache convention.

```
@Path("/api/greetings")
public class GreetingResource {

    @GET
    public List<Greeting> list() {
        return Greeting.listAll();
    }

    @POST
    @Transactional
    public Response create(GreetingRequest request) {
        Greeting greeting = Greeting.of(request.name());
        greeting.persist();
        return Response.status(Response.Status.CREATED).entity(greeting).build();
    }
}
```

Your generated types will not match this snippet character for character. Check the behaviors: entity extends PanacheEntity (or uses a Panache repository), POST is @Transactional, and GET does not open a write transaction it does not need.

If application.properties now contains quarkus.datasource.jdbc.url **without** a %prod. prefix, Dev Services stay off. Ask Bob to remove the default URL so only the prod profile (next section) points at an explicit database.

9.3 Verify POST then GET

Use the URLs Bob reported if they differ.

Create a greeting:

```
curl -i -X POST http://localhost:8080/api/greetings \
-H "Content-Type: application/json" \
-d '{"name":"Lab"}'
```

You should see HTTP 201 and a body that includes "name":"Lab" and a message such as Hello, Lab, plus a numeric id.

IBM TechXchange

List greetings:

```
curl http://localhost:8080/api/greetings
```

The array must include the row you just posted. Fetch that row by the numeric id returned by POST (replace 1 below), then check a missing ID:

```
curl -i http://localhost:8080/api/greetings/1
curl -i http://localhost:8080/api/greetings/999999
```

The first request returns HTTP 200 and the same row. The second returns HTTP 404 if that ID does not exist. These rows are stored in PostgreSQL. With the lab's drop-and-create schema setting, restarting the application recreates the tables and clears their contents.

SUCCESS: Stop and verify

POST returns 201 with an id. GET /api/greetings and GET by that id return the row. A missing id returns 404. MCP tests pass.

If POST fails with a transaction or persistence error, ask Bob:

```
POST /api/greetings failed. Call quarkus_skills for Panache, then
devui-exceptions_getLastException via quarkus_callTool, fix the issue,
re-run tests via MCP, and clear the exception.
```

10 Containerize with Podman

Time: 22 minutes **Mode:** Agent (or Advanced)

Dev Services exist only in dev and test. A container image runs as **prod**. Prod does not start Postgres for you, so the image must talk to a database you run explicitly.

This section follows the same build-and-run idea as [Containerizing Quarkus applications and deploying them with Kubernetes](#): add a container-image extension, build with Podman, run the image. This lab stops at Podman on the VM. Kubernetes manifests are optional further reading, not a lab requirement.

You will:

1. Add quarkus-container-image-podman through MCP while dev mode is running, then stop dev mode.
2. Configure the image and a **prod-only** datasource.
3. Build localhost/greeting-service:lab.
4. Run Postgres and the app on one Podman network.
5. Repeat the POST/GET checks against the mapped port.

10.1 Build the image and prepare prod config

Paste this prompt:

```
Containerize greeting-service for Podman using Quarkus container-image support.
```

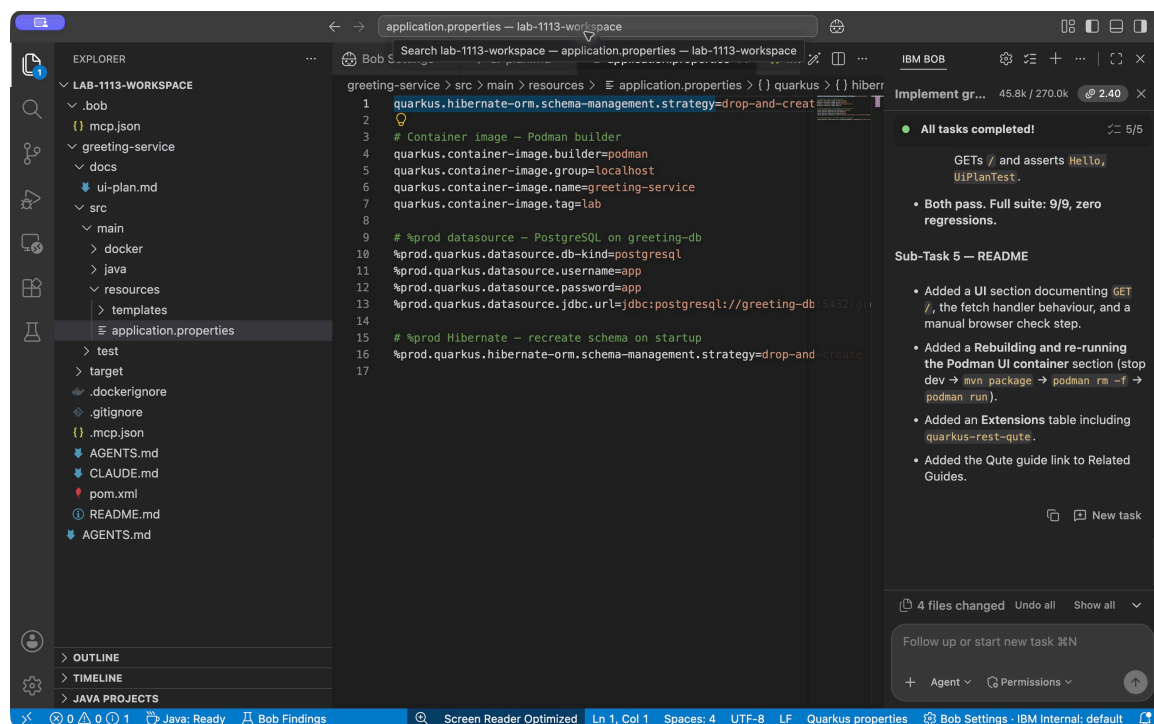
Steps:

1. Call `quarkus_skills` and `quarkus_searchDocs` for `container-image` / `Podman`
2. Add `quarkus-container-image-podman` through the MCP extension tool while dev mode is running (not Docker, not Jib)
3. Stop the app with `quarkus_stop` before packaging
4. Set:
`quarkus.container-image.builder=podman`
`quarkus.container-image.group=localhost`
`quarkus.container-image.name=greeting-service`
`quarkus.container-image.tag=lab`
5. Add `%prod` datasource config for PostgreSQL:
 - host/service name: `greeting-db`
 - database: `greetings`
 - username/password: `app / app`
 - jdbc url: `jdbc:postgresql://greeting-db:5432/greetings`
 - `%prod.quarkus.hibernate-orm.schema-management.strategy=drop-and-create`Keep Dev Services working in dev: do not set `jdbc.url` outside `%prod`
6. Keep Quarkus 3.39.5. Package and build the JVM image (not native) with `mvn package -Dquarkus.container-image.build=true`
7. Give me exact podman commands to:
 - create a network `greeting-net`
 - run `postgres:16` as `greeting-db` on that network
 - wait for `pg_isready` in `greeting-db` before starting the app
 - run `localhost/greeting-service:lab` as `greeting-app`, publish 8080
 - `curl POST and GET /api/greetings`

Do not install Docker.

Watch for:

- `quarkus_stop` before the package/image build.
- Extension `quarkus-container-image-podman` in the project.
- Image coordinates in `application.properties` (or `application.properties` plus profile files).
- `%prod.` prefixes on the datasource URL. If those properties are unprefixed, the next `quarkus_start` will skip Dev Services.



Container image properties

The prod schema setting drop-and-create creates the tables for this lab and clears existing rows on every app startup. A real production app uses Flyway or Liquibase and schema-management.strategy=none (or validate). Do not copy drop-and-create into a production system.

10.2 Review prod configuration

application.properties should contain entries equivalent to:

```

quarkus.hibernate-orm.schema-management.strategy=drop-and-create

# Container image (Podman)
quarkus.container-image.builder=podman
quarkus.container-image.group=localhost
quarkus.container-image.name=greeting-service
quarkus.container-image.tag=lab

# Production datasource (Dev Services stays active in dev/test)
%prod.quarkus.datasource.db-kind=postgresql
%prod.quarkus.datasource.username=app
%prod.quarkus.datasource.password=app
%prod.quarkus.datasource.jdbc.url=jdbc:postgresql://greeting-db:5432/greetings
%prod.quarkus.hibernate-orm.schema-management.strategy=drop-and-create

```

The JDBC host greeting-db is a **container name** on the Podman network, not localhost. From inside the app container, localhost is the app itself.

10.3 Run Postgres and the app

Use Bob's commands if they differ in image tag or ports. One working sequence is:

```
podman network create greeting-net

podman run -d --name greeting-db --network greeting-net \
  -e POSTGRES_USER=app \
  -e POSTGRES_PASSWORD=app \
  -e POSTGRES_DB=greetings \
  docker.io/library/postgres:16

until podman exec greeting-db pg_isready -U app -d greetings; do
  sleep 1
done

podman run -d --name greeting-app --network greeting-net \
  -p 8080:8080 \
  localhost/greeting-service:lab
```

After the app starts, check the image and both containers:

```
podman images | grep greeting-service
podman ps
```

Both greeting-db and greeting-app should be running. If the app container restarts, inspect logs:

```
podman logs greeting-app
```

A typical failure is Connection refused to greeting-db: wrong network, wrong hostname, or Postgres still starting. See [Podman errors](#) and [Container cannot reach PostgreSQL](#).

Check Bob's README edits too: endpoint examples must use /api/greetings, and dev startup must use MCP. Review generated commands before approving them.

10.4 Prove the containerized API

```
curl -i -X POST http://localhost:8080/api/greetings \
  -H "Content-Type: application/json" \
  -d '{"name": "Container"}'

curl http://localhost:8080/api/greetings
```

Replace 8080 if you published a different host port.

SUCCESS: Stop and verify

podman images lists localhost/greeting-service:lab. podman ps shows the database and the app. POST/GET against the published port persist a greeting.

IBM TechXchange

If port 8080 is still held by a leftover dev-mode process, stop it (quarkus_stop or identify the process) and recreate the app container.

10.5 Optional: Plan a Qute UI

Time: ~15 minutes if you are ahead

This stretch is not required for the lab outcome. It practices Bob **Plan** mode: write a bounded plan, start a **new chat**, then implement.

Before starting this stretch, stop the app container so dev mode can use port 8080:

```
podman stop greeting-app
```

Leave greeting-db available for the later container check. Dev mode uses its own Dev Services database.

1. Switch to **Plan** mode.
2. Paste:

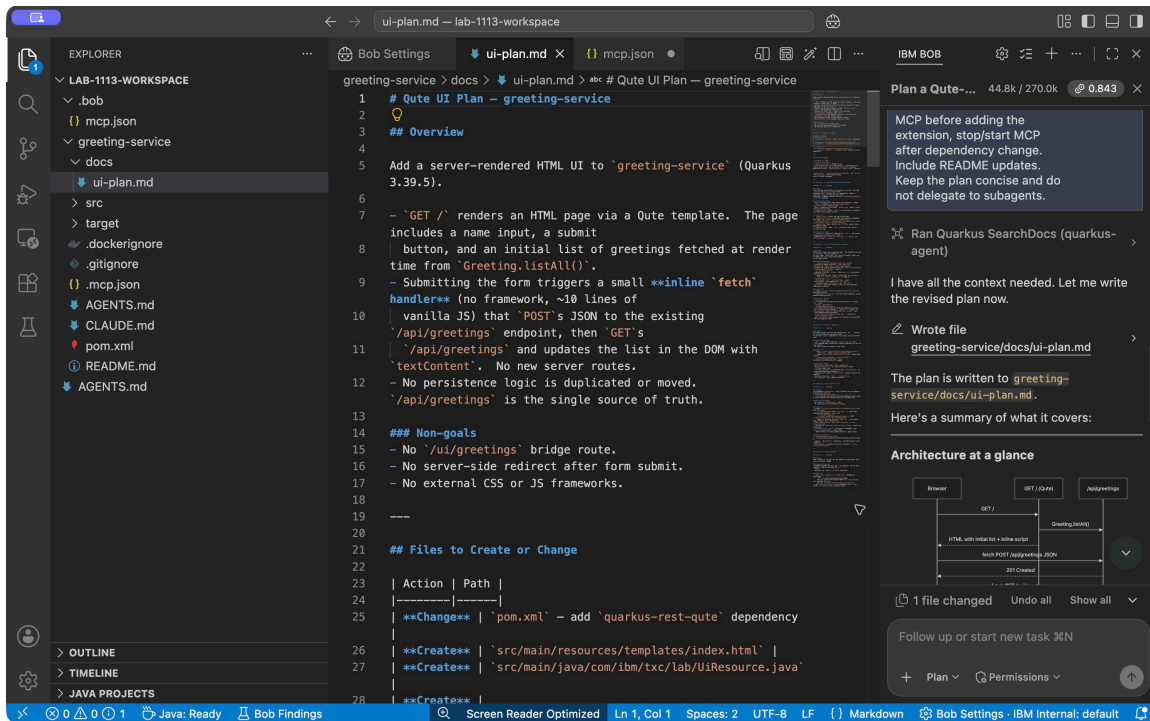
```
Plan a Qute-based UI for greeting-service. Do not implement yet.
```

```
Goals:
```

- GET / shows an HTML page with a form: text input for name, submit button
- Submitting sends JSON to POST /api/greetings, then GET /api/greetings refreshes the list; use a small fetch handler and textContent, no JS framework
- Keep the initial HTML server-rendered by Qute; do not add a second write endpoint
- Keep everything in this Quarkus app (Qute templates under src/main/resources/templates)
- List files to create or change, extensions needed, and a test plan

```
Write the plan to greeting-service/docs/ui-plan.md only.
```

3. Open greeting-service/docs/ui-plan.md and confirm the scope: Qute in this app, no separate SPA, no extra database product.



Plan mode with ui-plan.md

4. Click + (**New Task; New chat** in some builds) so planning discussion does not fill the implementation context. See [Delegate bounded implementation tasks](#).

5. Switch to **Agent** mode and paste:

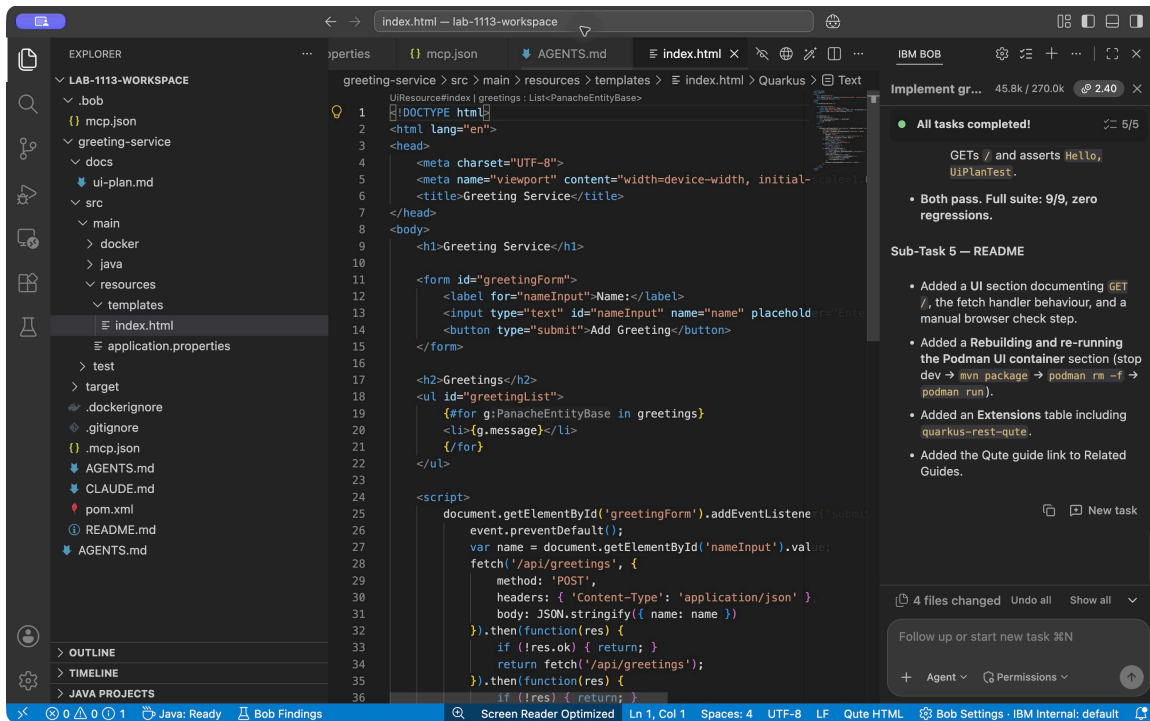
Implement greeting-service/docs/ui-plan.md exactly.

Rules:

- Call quarkus_skills separately for Qute and REST before coding; if no Qute skill is available, use quarkus_searchDocs for Qute integration guidance
- Use Qute templates; server-side render the page
- Reuse POST /api/greetings and GET /api/greetings
- Add or update tests; run tests via Quarkus MCP tools
- Start with quarkus_start, then add extensions through MCP and stop/start through MCP after dependency changes
- Keep Quarkus 3.39.5
- Update README.md with MCP startup instructions for dev and the stop, rebuild, and recreate steps needed to get this UI into the Podman image

6. Open <http://localhost:8080/> (or the port Bob reports), submit a name, and confirm the greeting appears. Reload the page and check that the stored greeting is still shown.

The image built earlier does not contain the new UI. To test the UI in Podman, ask Bob to stop dev mode with `quarkus_stop` and run `mvn package -Dquarkus.container-image.build=true` again. Recreate greeting-app from the rebuilt image on greeting-net using the same run command, then repeat the browser check. With drop-and-create, the app startup clears the lab database rows.



Quite template in the editor

Greeting Service

Name:

Greetings

- Hello, Quite Lab

Browser showing the Quite UI

For more information on Plan mode, see [Plan and implement complex features](#).

11 Close the loop

Time: 6 minutes

11.1 Recall

Answer this before you expand the note:

Which of these must go through Quarkus MCP in this lab, and which is a reasonable Maven command after 'quarkus_stop'?

- Creating the project
- Starting dev mode
- Running unit tests while the app is in dev mode
- Building the container image

NOTE: Answer

Create, start, and tests go through MCP (quarkus_create, quarkus_start, quarkus_callTool / Dev MCP test tools). Packaging the image with mvn package -Dquarkus.container-image.build=true (or the equivalent Bob runs after stop) is the Maven step this lab allows. Never run mvn clean while dev mode is still running.

Second check: *Why did '/api/greeting' not need Postgres, and why did the containerized '/api/greetings' fail if you only started the app image?*

NOTE: Answer

The query-parameter endpoint computed a string in memory. The collection endpoint reads and writes a Panache entity. Prod does not start Dev Services, so the app container needs a reachable PostgreSQL on the same Podman network and %prod JDBC settings that use that hostname.

11.2 Artifacts

Artifact	Location
MCP configuration	.bob/mcp.json
Bob instructions	AGENTS.md (workspace root)
Quarkus application	greeting-service/
Persisted greetings API	POST / GET /api/greetings
Container image	localhost/greeting-service:lab
Optional UI plan	greeting-service/docs/ui-plan.md

11.3 What changed

Bob is the SDLC partner in the editor. Quarkus Agent MCP is the expert toolchain for lifecycle, skills, docs, and tests. You remain the verifier: curl, podman ps, and tests you actually see pass.

IBM TechXchange

The production-shaped loop is the same one the abstract promised: create REST, persist data, run the app in a container. Dev Services keep development moving. Explicit %prod config and a second container are what make the image honest.

11.4 Further reading

- [Quarkus Agent MCP](#) — tool list, IBM Bob mcp.json notes, PATH troubleshooting
- [Container images \(Quarkus\)](#) — Podman builder and image coordinates
- [Hibernate ORM with Panache](#)
- [Dev Services](#)
- [Containerizing Quarkus applications and deploying them with Kubernetes](#) — next step after this lab if you want Kubernetes manifests

11.5 Need help?

See [Troubleshooting](#) for sign-in, SDKMAN/Maven, MCP, Dev Services, ports, Podman, and recovery prompts.